

02-614 Week 4

String Algorithms

Aidan Jan

September 15, 2026

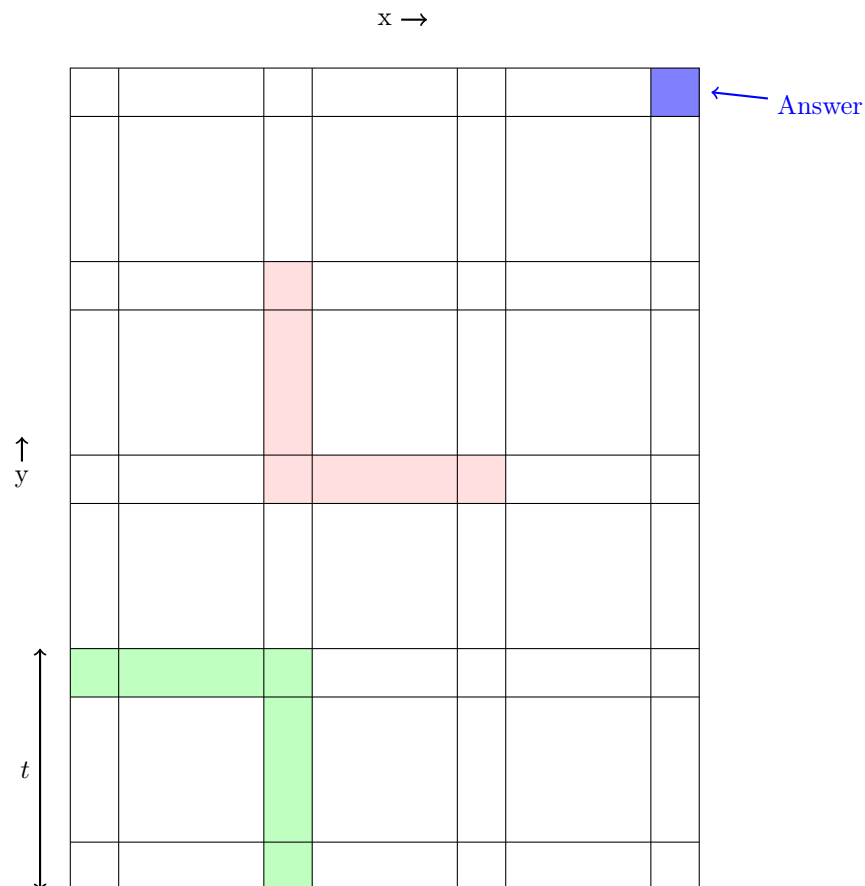
Four Russian's Speedup

There is a faster way to do edit distance than Hirschberg's algorithm that also keeps the same space complexity.

- **Goal:** Edit distance faster than $O(m, n)$ with no increase in space (asymptotically).

Idea 1:

Imagine we can tile our dynamic programming table with tiles of size t . Instead of calculating every cell individually (and take up mn time to do so), we can compute in larger chunks.



- Each tile overlaps the other tiles by 1 row and 1 column on each side.

- We start from the bottom left corner (which we call cell $(0,0)$). Consequently, our answer is the top right.

First, we have to make a few assumptions:

- $m = n$, or in other words, $|x| = |y|$.
- Our tiles exactly tile the DP matrix.

Idea 2:

With this tile setup, only the edges of the tiles matter.

- If we know the edges, then we don't need the centers of the tiles.

We want:

- A function $f(\text{red shaded edges}, x', y') \rightarrow \text{green shaded edges}$. In other words, take in the bottom and left edges of each tile and output the values of the top and right edges.

It turns out that we can do this using the Hirschberg or the Naïve dynamic programming algorithm. Therefore, a tile would take $O(t^2)$ time. Can we compute that faster?

Computing f Faster

Lemma: In the edit distance (number of operations) DP matrix D , adjacent cells in a row, column, or diagonal differ by at most 1.

		0	-1	-1	-1	← Output offset Vector
1		$c+2$	$c+1$	c	$c-1$	
1		$c+1$	$c+1$	c	$c-1$	
0		c	$c+2$	$c+3$	$c+4$	← Violation of the Lemma
0		c	$c+1$	c	c	
		0	1	-1	0	← Input offset Vector

Idea 3:

Represent the **red shaded edges** and **green shaded edges** as offset vectors.

- In the example above, we define the bottom offset vector at $(0, 1, -1, 0)$, since those are the differences between each of the values in the cells along the bottom rows
- Since every tile represents matching a substring between x and y and the bottom row and left columns represent the previous alignment, the best alignment (the output) always remains the same if the input offset vectors and the characters in the string are the same.
- This means we can precompute every f value and do lookups to fill in the edges of tiles on the DP table.

Idea 4:

Precompute every possible f value.

- (1) There are $3^{2(t-1)} \cdot |\Sigma|^{2t}$ possible inputs to f .
 - $3^{2(t-1)}$ comes from the fact that offset vectors must contain either $\{-1, 0, 1\}$ in each position, and there are a total of $2(t-1)$ positions along the left and bottom sides of a tile.
 - The $|\Sigma|^{2t}$ comes from the number of possible substrings that could be in x and y , for $2t$ characters total, and each character has $|\Sigma|$ choices.
- (2) Computing all possible values takes $O((3|\Sigma|)^{2t} \cdot t^2)$. (The t^2 is from the DP algorithm to fill in the cells of a tile once.)

However, the trick here is that t is an arbitrary tile size value, which we are able to set. We pick $t = \frac{1}{2} \left(\log_{3|\Sigma|} n \right)$.

- Now, if we plug in for t , our time complexity becomes $O(n(\log n)^2)$.

This pre-computation now takes less time than the rest of the problem because we only do it once.

Storing f_s

We make a tree that encodes the input and stores the output at the leaves.

- Start at the root
- The first $2(t-1)$ layers are all the possible offset vector combinations
- The next $2t$ layers are possible substrings.
- The output offset vectors (i.e., the top and right sides of each tile) are the data stored at the leaf nodes.

With this, searching up an f value is linear time, since our tree is $O(t)$ layers deep.

- In this case, the tree acts similarly to a hash map except without the $O(1)$ lookup. It maps an input key (input offset vectors and substrings) to an output value (output offset vectors).
- We don't use a hash map because the number of keys we have is exponential - a hash map would run into bucketing problems and become linear time (relative to the number of total possible inputs, i.e., exponential) lookup.
- The search time for a tree of $O(t)$ height would be $O(t)$, which is faster than all the other parts of the algorithm anyways.

Total Running Time

- We have n^2/t^2 tiles.
- It takes $O(t)$ to solve each tile by using lookups in the tree.
- The preprocessing takes $O(n \log^2 n)$ time.

In total, we have $O(\frac{n^2}{t^2} \cdot t + n \log^2 n)$, which simplifies to $O(\frac{n^2}{t} + n \log^2 n)$. Since the size of t we determined is equal to $\log n$, we can substitute that in, and drop the second term. (The $\frac{n^2}{\log(n)}$ term dominates.)

$$\boxed{O\left(\frac{n^2}{\log n}\right)}$$